

AM234/BME234 Homework 3

Due: Friday June 5, 2026 at 11:59 pm

1. (20 pt) Random forests use ensemble learning to reduce the variance of an individual decision tree by averaging predictions across multiple decision trees. We will explore this further in this problem.

(a) (4 pt) Consider a set of N uncorrelated variables, $X_1, \dots, X_N$ with mean μ and variance σ^2 . (In the context of ensemble learning, X_i is the prediction made by classifier i .) What is the variance of the average of these variables?

(b) (4 pt) Random forests use bagging (short for **bootstrap aggregating**), an ensemble learning method in which each learner is trained on a separate bootstrapped dataset.

To construct a bootstrapped dataset, we randomly sample M data points (with replacement) from the original training set of size M . In this process, some points may be chosen multiple times, while others may not be chosen at all. The points that are not chosen are denoted as out-of-bag (OOB) samples for that decision tree.

Why do we expect that 37% of samples will be OOB for large values of M ?

Hint: $\lim_{n \rightarrow \infty} (1 - \frac{1}{n})^n = \frac{1}{e}$

(c) (2 pt) To determine the appropriate number of decision trees in a random forest model, we could use cross-validation. An alternative is to examine the OOB error. Explain what the OOB error is and how it can be used to choose the appropriate number of decision trees.

(d) (10 pt) In part a, we found that averaging reduces variance for uncorrelated classifiers. Predictions made by different decision trees, however, will not be completely uncorrelated, but reducing correlation among decision trees will reduce the final variance. To illustrate this, consider a set of correlated random variables $X_1, \dots, X_N$. Similar to part i., they each have mean μ and variance σ^2 . However, this time, $0 \leq \text{Cov}(X_i, X_j) \leq \delta^2$ for all $i \neq j$. Find an upper bound for the variance of the average of the X variables.

2. (10 pt) Neural network parameters.

(a) (4 pt) Suppose that you have a fully-connected neural network with the following dimensions for each layer:

- input: 32

- hidden layer 1: 128
- hidden layer 2: 64
- output layer: 10

Assume that each neuron also contains a bias term. How many parameters are in this neural network?

(b) (6 pt) Suppose that a convolutional neural network consists of the following layers:

- a convolutional layer with 100 filters of size 3x3
- ReLU activation
- a max pooling layer of size 2x2
- a convolutional layer with 50 filters of size 2x2
- ReLU activation
- a max pooling layer of size 3x3

Further, assume that each convolutional filter also contains a bias term. How many parameters are in this neural network?

Hint: For more details on the architecture of convolutional neural networks, we recommend this guide: <https://cs231n.github.io/convolutional-networks/>

3. (50 pt) In this problem, you will train a machine learning algorithm to predict the pathogenicity of missense variants. Your model will be trained on variants from ClinVar, a publicly accessible database of human variants with pathogenicity assertions.

You should submit your code in a file named `problem_3.py`.

(a) (4 pt) First, construct a dataset using the data from ClinVar. We have already preprocessed the raw ClinVar data for you and extracted only missense variants to simplify this process. The processed data is located in `clinvar_missense.vcf`. However, you should not construct a dataset using all variants in this file as not all of them have benign or pathogenic clinical significances.¹

To isolate only benign and pathogenic missense variants, exclude all variants that do not have a *Benign* or *Pathogenic* **CLNSIG** value. After this filtering, how many benign and pathogenic variants remain? Assign pathogenic variants a class label of 1 and benign variants a class label of 0.

Note: Do not include variants with *Likely_benign* or *Pathogenic/Likely_pathogenic* clinical significances. The significance string should exactly match either *Benign* or *Pathogenic*.

(b) (3 pt) Randomly split the dataset from part (a) into a training set and a validation set such that the training set contains 80% of the variants. How many benign and pathogenic variants are in each split?

(c) (7 pt) Now you are ready to construct features for each variant. We will first include two features that quantify the harmfulness of mutating a gene: "RVIS" and "o/e". RVIS (Residual Variation Intolerance Score) ranks genes according to whether they have more

¹<https://www.ncbi.nlm.nih.gov/clinvar/docs/clinsig/>

or less common functional (i.e. missense or nonsense) variation relative to the amount of neutral variation.

Intuitively, the RVIS score captures the following phenomenon. If a gene contains 1000 variants from a sequencing study of healthy individuals and 900 of them are common (i.e. present with an AF > 1%) missense variants, then mutations in this gene are tolerated. On the other hand, if the gene contains only 10 common missense variants out of the 1000 observed, then mutations in this gene are not tolerated.

The RVIS data are provided to you in the file `RVIS_Unpublished_ExACv2_March2017.txt`. The important columns in this file are the `CCDSr20` column, which contains the gene symbol, and the `%RVIS[pop_maf_0.05%(any)]` column, which contains the RVIS score—normalized as a percentile—for that gene.

Find the RVIS score for each variant in your dataset. To determine which gene each variant overlaps, examine the `GENES` info field in the VCF. For genes without an RVIS score, assign a default percentile of 50, and for variants that overlap multiple genes, take an average of their RVIS scores.

Plot the histogram of RVIS values for the pathogenic variants and benign variants in your dataset. Based on the plot, do you think RVIS is a useful feature to add to the model?

- (d) (7 pt) The o/e (observed/expected) ratio is a similar metric computed using newer data than RVIS (gnomAD instead of ExAC) and more sophisticated statistical models. Like RVIS, the o/e ratio provides a score for each gene.

Download the o/e data from https://storage.googleapis.com/gcp-public-data--gnomad/release/2.1.1/constraint/gnomad.v2.1.1.lof_metrics.by_gene.txt.bgz. The important columns in this file are the `gene` column and the `oe_lof_upper_rank` column, which provides the o/e rank for the gene.

Similar to the previous part, find the o/e score for each variant in your dataset. Plot the histogram of o/e values for the pathogenic variants and benign variants. Based on the plot, do you think the o/e metric is a useful feature to add to the model?

Hint: Similar to the previous part, you will need to handle the following two edge cases: (a) genes without o/e scores and (b) variants overlapping multiple genes.

- (e) (7 pt) We will now include a variant-level feature in the model. phastCons is a two-state Hidden Markov Model (HMM) that divides the genome into conserved and nonconserved segments using multiple sequence alignments and phylogenetic data. The phastCons score for each nucleotide describes the probability that the nucleotide is part of a conserved region and can be incorporated as a feature to determine whether a missense mutation is likely to be damaging.

The nucleotide-level phastCons scores are stored in a bigWig file, which can be downloaded from <https://hgdownload.cse.ucsc.edu/goldenPath/hg38/phastCons100way/hg38.phastCons100way.bw>. (Warning: This file is more than 5 GB!) To read bigWig files, use the `pyBigWig` library. Documentation can be found on its Github repo: <https://github.com/deeptools/pyBigWig>.

Plot the histogram of phastCons values for the pathogenic variants and benign variants in your dataset. Do you think this is a useful feature to add to the model?

- (f) (15 pt) Train a Random Forest model using these three features. Implement and perform

cross-validation on the training set in order to choose values for the required hyperparameters. List the values of these chosen hyperparameters. Plot an ROC curve on the validation set and display the AUC (AUROC).

- (g) (7 pt) Use your trained model to make predictions for each of the variants provided in the test set: `test_set.vcf`. Upload a file called `test_set.predictions` along with your code file that contains the predicted probability of a variant being pathogenic (the probability for each variant in the VCF file must be on a separate line).